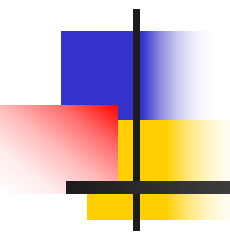


C语言程序设计

The C Programming Language



第8章 指针

华中科技大学计算机学院

卢萍

8.1 指针的概念

- 变量占有有一定数目(根据类型)的连续存储单元;

`short x;`

`char a[5];`

- 变量的连续存储单元首地址称为**变量的地址**。

`&x,     a <= => &a[0]`

- 思考：用什么类型的变量来保存地址数据？**

地址		变量名
0xFEBC		x
0xFEBD		
0xFEC0		a[0]
0xFEC1		a[1]
0xFEC2		a[2]
0xFEC3		a[3]
0xFEC4		a[4]

指针变量

➤ **指针**: 变量的地址

&x: 指针常量

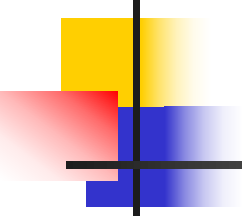
p: 指针变量

➤ **指针变量**: 存放地址数据的变量。

指针变量也是一种变量，也要占用一定的内存单元。

指针的特殊之处在于它存放的是另一个变量所占存储单元的起始地址。

地址		变量名
0xFEBC	1	x
0xFEBC	0	
	2	y
	0	
		p
0xFEC0		
0xFEC1		
0xFEC2		
0xFEC3		
0xFEC4		



指针变量的声明

数据类型 *标识符;



所指(即所指向)变量的数据类型

```
short x=1, y=2, a[10], *p;
```

```
p=&x;
```

```
p=a;
```

```
p=a[0]; // X
```

思考: 为什么使用指针?

间接访问

➤ **直接：**通过变量名存取（或访问）变量

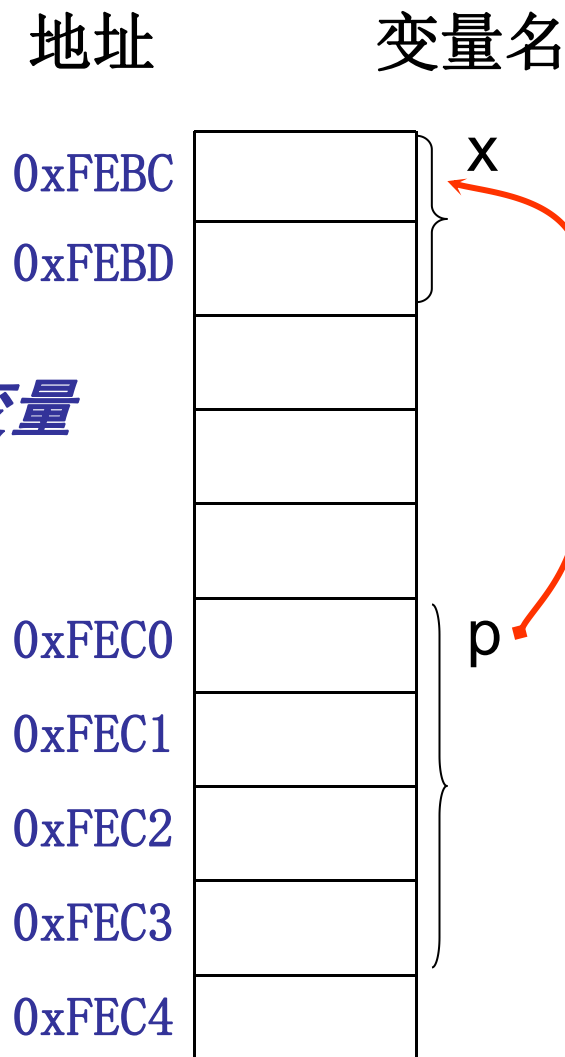
$x=0x1234$; $printf("%x" , x);$

➤ **间接：**通过变量的地址（即指针）存取变量

$p=\&x$;

$printf("%x" , *p);$

先访问 p ，得到 x 的地址，
再通过该地址找到它所指向的单元中的值。



指针运算符 *

***操作数** /* 返回 操作数所指地址处的变量值。*/

short x=1, y=2, *p; /* *p是short型指针变量* */

p=&x; /* *将x的地址给p，即指针p指向x* */

y=*p; /* **p==>x* */

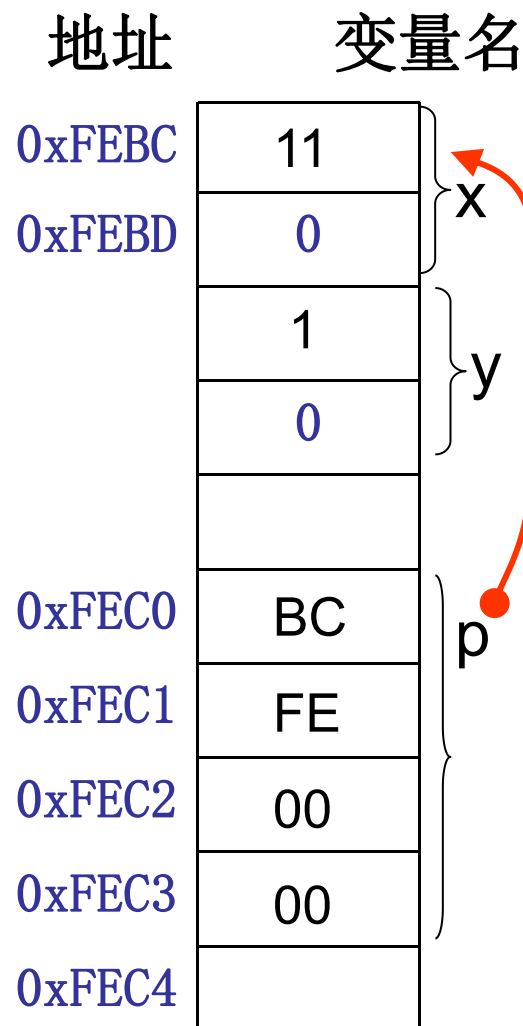
p += 10; / *x+=10* */

printf("x=%hd,y=%hd",x,y) ;

输出？

x=11,y=1

思考：为什么指针有类型？



悬挂指针

- 指出下面程序段的错误:

```
int x,*p; /* 说明p是一个整型指针变量, 其值不确定. */
```

```
x=25;
```

```
*p=x; /* 使用悬挂指针, 错! */
```

```
scanf("%d", p); /* 使用悬挂指针, 错! */
```

- 指针的声明只是创建了指针变量, 获得了指针变量的存储, 但并没有给出指针变量指向那个具体的变量, 此时指针的值是不确定的随机值, 指针处于“无所指”的状态。称为悬挂指针（野指针）。
- 不能使用悬挂指针

指针的移动操作

(+, -, ++, --, +=, -=)

指针的移动：指针在原有位置的基础上，通过加一个整数实现指针的**前移**（地址增大的方向）或者通过减一个整数实现指针的**后移**。

$p \pm k$

↑ ↑

指针 整型

p 前移/后移 k 个元素

指针的移动

已知声明如下，指出下面表达式的值。

(各题的表达式相互无关)

```
int x[5]={1, 3, 5, 7, 9}, *px=&x[1];
```

(1) ++*px 4

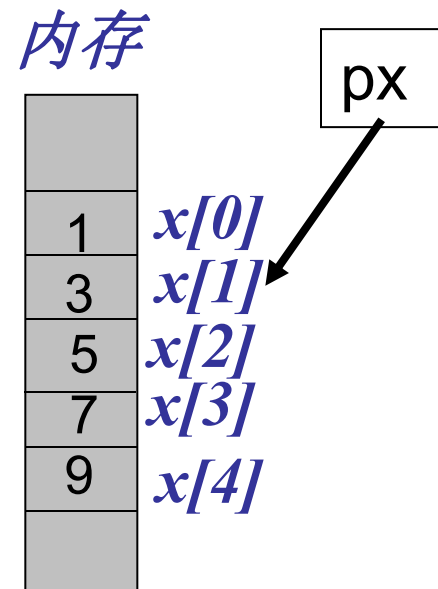
(2) *++px 5

(3) *(--px) 1

(4) *(px--)

(5) *px++ 3

(6) px+=2, *px 7



指针的移动和类型有关

```
int x=0x1234ABCD, a,b,*p1;
```

```
char *p2;
```

```
p1=&x; p2=(char *)&x;
```

```
a=*p1;
```

```
b=*p2;
```

```
printf("%x,%x",a,b) ;
```

```
b=*(p2+1);
```

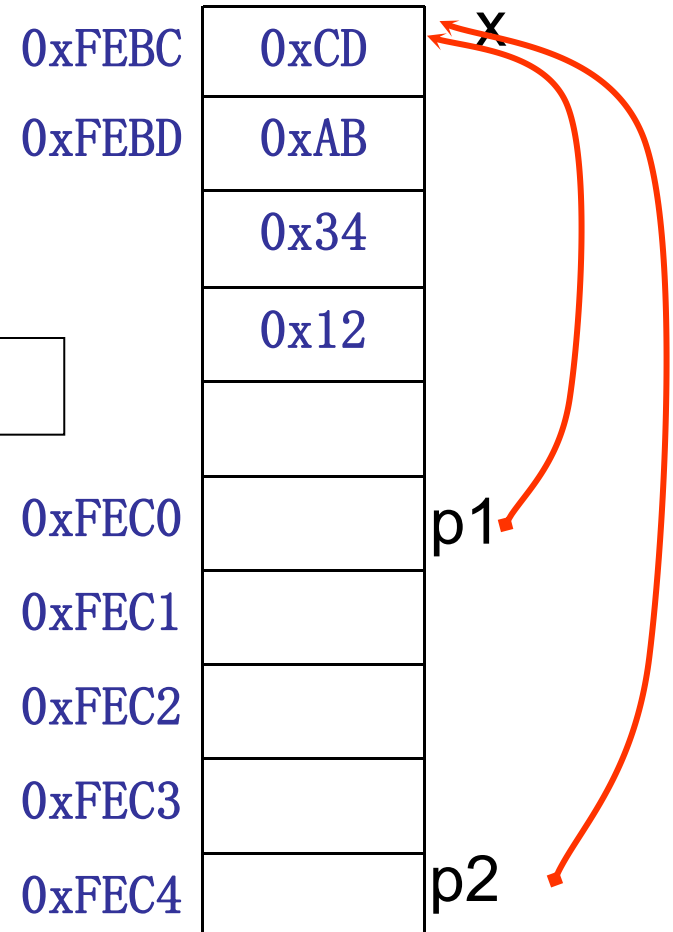
```
printf("%x",b) ;
```

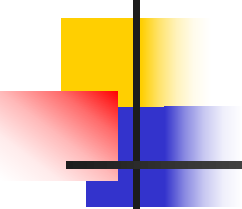
1234ABCD,CD

AB

地址

变量名





指针的赋值运算

(1) 同类型的指针可以直接赋值。如：

```
int a[3]={1, 2, 3}, x, *p, *q;
```

```
p=a; q=&x;
```

(2) 不同类型的指针必须使用类型强制转换再赋值。

```
long x;
```

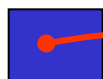
```
char *p;
```

```
p = (char *)&x;
```

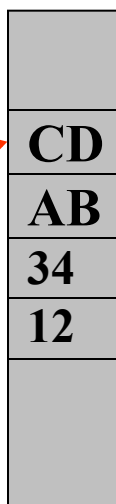
指针的应用

例：一个长整型数占4个字节，其中每个字节又分成高4位和低4位。试从长整型数的低字节开始，依次取出每个字节的高4位和低4位并以十六进制字符的形式进行显示。

char *p



内存



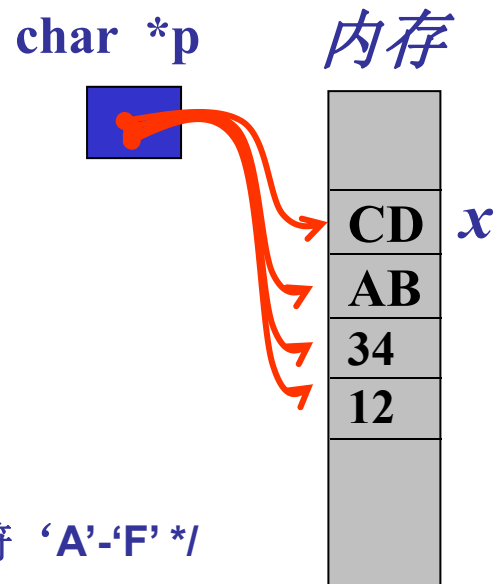
$x=0x1234ABCD$

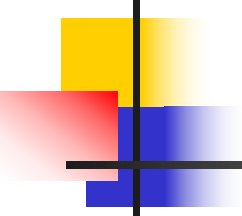
取低4位: $(*p)\&0x0f$

取高4位: $(*p>>4)\&0x0f$

指针的应用

```
#include <stdio.h>
int main(void)
{
    long x=0x1234ABCD,k;
    char *p=(char *)&x;
    char up_half,low_half;
    for(k=0;k<4;k++) { /* 依次取每字节 */
        low_half=(*p)&0x0f; /* 取低4位 */
        if(low_half<10)
            low_half+='0'; /* 低4位转成字符 '0'-'9' */
        else
            low_half=(low_half-10)+'A'; /* 低4位转成字符 'A'-'F' */
        up_half=(*p>>4)&0x0f; /* 取高4位 */
        if(up_half<10)
            up_half|='0';
        else
            up_half=(up_half-10)+'A';
        printf("%c  %c\n",up_half,low_half);
        p++; /* 指向整型数x下一个字节 */
    }
    return 0;
}
```





指针练习一

1、检测机器的大小端模式

int checkEndion(void);// 返回1小端，0大端

2、定义宏提取一个16位数据的高八位和低八位

#define BYTE0(hdata) _____

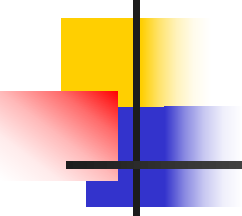
#define BYTE1(hdata) _____

3、将int数的最高字节和最低字节交换。

4、将32位整数IP转为点分十进制输出（实验2中第6道编程题）

5、32位本机字节序与网络字节序之间的转换，以 host to network为例，hton(0x1234)值为0x34120000

unsigned long hton(unsigned long h);



8.2 指针参数

传值调用：形参的修改无法影响实参变量的值。

```
#include <stdio.h>
```

```
void swap(int x, int y)
```

```
{ int t;
```

```
    t=x; x=y; y=t;
```

```
}
```

```
int main(void)
```

```
{ int a=3, b=5;
```

```
    swap(a, b);
```

```
    printf( "a=%d, b=%d\n", a, b);
```

```
    return 0;
```

```
}
```

指针作函数参数

传址调用：以指针作为函数的参数实现变量值的交换。

```
#include <stdio.h>
```

```
void swap(int *px, int *py)
```

```
{  int t;
```

```
    t=*px;*px=*py;*py=t;
```

```
}
```

```
int main(void)
```

```
{  int a=10, b=20, c=30;
```

```
    swap(&a, &b);
```

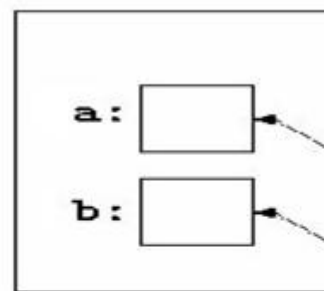
```
    swap(&b, &c);
```

```
    printf( "a=%d, b=%d, c=%d\n ", a, b, c );
```

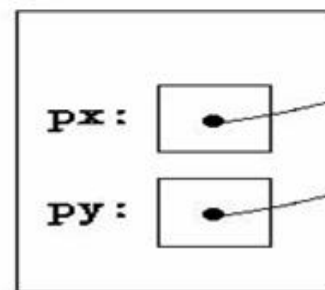
```
    return 0;
```

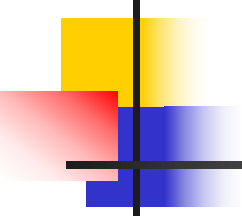
```
}
```

in caller:



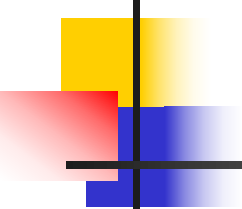
in swap:





指针作函数参数

- 改变主调函数中变量的值
- 使函数送回多个值



指针作输出参数

指针可作输出参数将值传回调用函数，使函数间接返回值，解决函数返回多值的问题。

```
// implicit returned values:
```

```
void sum(int x,int y,int *p)
```

```
{
```

```
    *p=x+y;
```

```
}
```

```
// the caller
```

```
int sum;
```

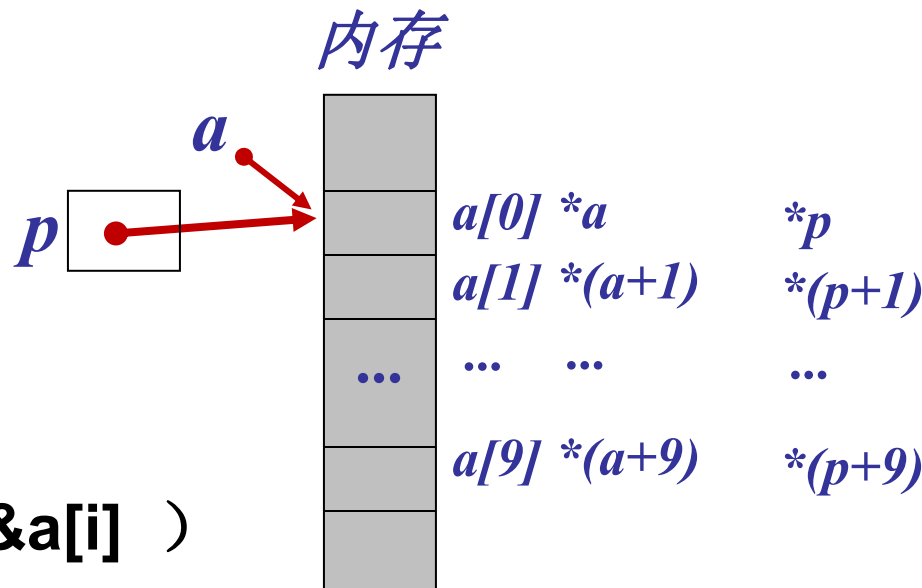
```
sum(3,4,&sum);
```

8.3 指针和一维数组

数组元素既可以用下标表示，也可以用指针表示。

```
int a[10], *p=a;
```

数组元素的表示



下标法 $a[i]$ (地址为 $\&a[i]$)

指针法 { 数组名 $*(a+i)$ (地址为 $a+i$)

指针变量 $*(p+i)$ (地址为 $p+i$)

数组元素的输入

已知: #define N 10

int a[N], *p, i;

p=a;

输入数组的全部元素

(1) for(i=0;i< N;i++)

scanf("%d", _____) ;

(2) for(;p<a+ N;p++)

scanf("%d", _____ **p**) ;

&a[i]

a+i

p+i

p++

a++

×

数组元素的输出

已知: `int a[10], *p, i;`

`p=a;`

输出数组的全部元素

`for(i=0;i<10;i++)`

`printf(“%d”, _____ );`

`for( ; p<a+10;p++)`

`printf(“%d”, _____ );`

`a[i]`
`*(a+i)`
`*(p+i)`
`*p++`

数组元素的输入和输出

输出数组*a*的全部元素(正确否 ?)

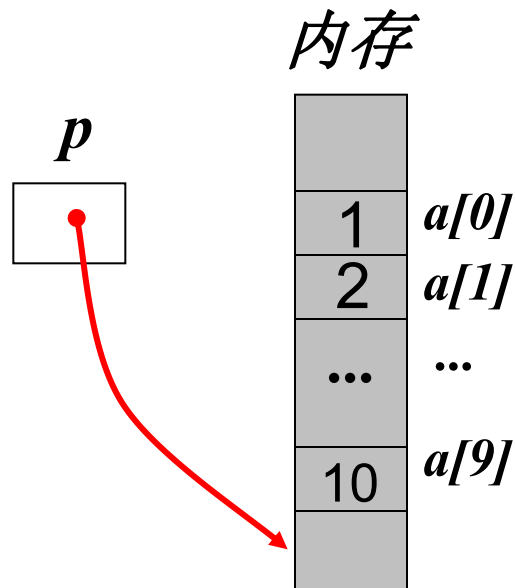
```
int a[10],*p , i;
```

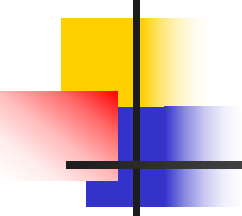
```
for( p=a, i=0; i<10; i++)
```

```
    scanf("%d", p++) ;
```

```
while(p<a+10)
```

```
    printf("%d", *p++ ) ;
```





指针的关系运算

<, <=, >, >=, ==, !=

只限于同类型指针，

不同类型指针之间的关系运算被视为非法操作。

// 逆序输出数组a的全部元素

```
int a[10],*p=a ;
```

```
while(p<a+10)  scanf("%d", p++);
```

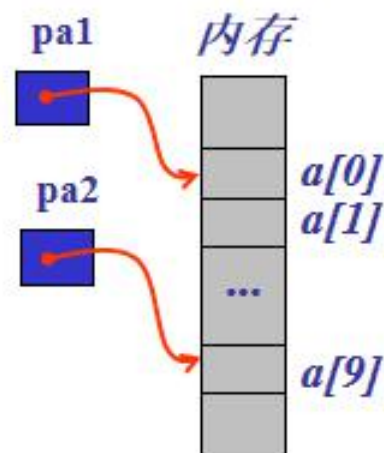
```
while( p>a)  printf("%d", *--p );
```

两个指针相减

两个指针可以相减

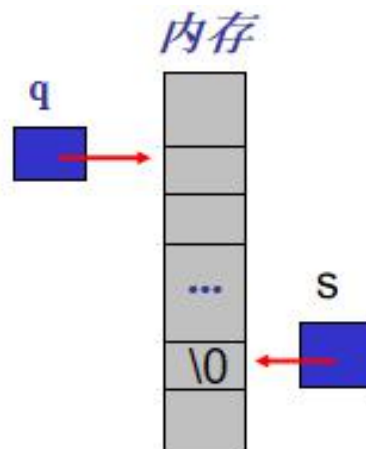
$pa2 - pa1$ /*等于所指元素的下标相减*/

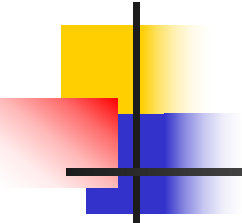
指向同一数组的元素



`int strlen(char s[ ]) /* 等价于 int strlen(char *s ) */`

```
{  
    char *q=s;  
    while(*s) s++;  
    return (s-q);  
}
```





常量指针

常量指针：就是指向常量的指针。

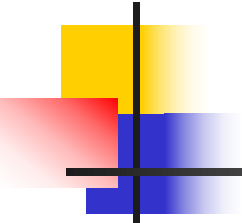
```
int a = 5, b = 6;
```

```
const int *p; /* p是常量指针，可以不初始化 */
```

```
p = &a;      /* 合法，可以修改指针本身的值 */
```

```
*p = 10;     /* 非法，不能通过p修改所指变量a的值 */
```

```
a = 8;       /* 合法，因为a不是常量 */
```



指针常量

指针常量：就是类型是指针的常量。

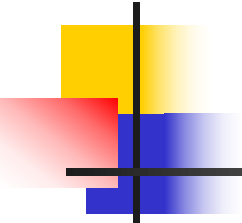
```
int a = 5, b = 6;
```

```
int *const p = &a; /* p是指针常量，必须初始化 */
```

```
*p = 10; /* 合法，可以使用p修改所指对象的值 */
```

```
p = &b; /* 非法，它不能再指向别的变量 */
```

```
int x[10]; /* x是指针常量 */
```



指向常量的指针常量

```
int a = 5, b = 6;
```

```
const int * const p = &a ;  /* p是指向常量的指针常量，  
                               必须初始化 */
```

```
*p = 10;    /* 非法，不能通过p修改所指变量a的值 */
```

```
p = &b;     /* 非法，指针本身的值也不能修改 */
```

一维数组参数的指针表示

```
void sort ( int a[ ], int n)
{
    .....
}
```

↑ 等价于 `int *a`

形参 { 不指定长度的数组
指针

实参 { 数组名（指针常量）
指向数组元素的指针变量

```

#include<stdio.h>
#define N 10
void BubbleSort ( int *a,int n) // 形参为指针
{
    int i, j, t;
    for (i=1; i<n; i++) /* 共进行n-1轮"冒泡" */
        for (j=0; j<n-i ; j++) /* 对两两相邻的元素进行比较 */
            if (a[j]>a[j+1] ) { t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
}
int main ( )
{ int x[N], i;
  printf (" please input %d numbers: \n ", N);
  for (i=0; i<N; i++) scanf(" %d ", &x[i]);
  BubbleSort (x , N ) ; // 实参为数组名
  printf (" the sorted numbers: \n ");
  for (i=0; i<N; i++) printf("%d ", x[i]);
  return 0;
}

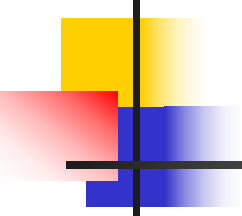
```

```

#include<stdio.h>
#define N 10
void BubbleSort ( int *a,int n) // 形参为指针
{
    int i, j, t;
    for (i=1; i<n; i++) /* 共进行n-1轮"冒泡" */
        for (j=0; j<n-i ; j++) /* 对两两相邻的元素进行比较 */
            if ( *(a+j)>*(a+j+1) ) {
                t=*(a+j); *(a+j)=*(a+j+1); *(a+j+1)=t;
            }
}

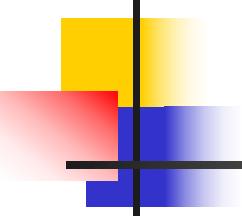
int main ( )
{ int x[N], i,*p=a;
  printf (" please input %d numbers: \n ", N);
  for (i=0; i<N; i++) scanf(" %d ", &x[i]);
  BubbleSort (p , N ) ; // 实参为指针变量
  ..... ;
}

```



高精度计算--- 超大整数的加法

高精度运算:是指参与运算的数(加数, 减数, 因子...) 范围大大超出了标准数据类型 (整型, 实型) 能表示的范围的运算。例如。求两个**100**位的数的和, 计算**100!** , 都要用到高精度算法。



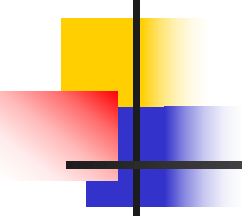
高精度计算主要解决的问题

高精度计算主要解决以下三个问题：

一、大数据（加数、结果）存储

二、运算过程

三、大数据的输入和输出



存储

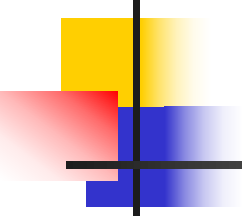
用数组存储，每个数组元素存储1位（可以优化！），有多少位就需要多少个数组元素。个位在 $x[0]$ ，十位在 $x[1]$, ...

用数组表示数的优点：每一位都是数的形式，可以直接加减；运算时非常方便。

主函数

```
#define N 200    /* 最大位数 */
int main(void)
{
    int x[N+1],y[N+1],z[N+2],len,i;

    printf("输入被加数: ");
    getBigNum(x,N);    /* 输入被加数 x */
    printf("输入加数: ");
    getBigNum(y,N);    /* 输入加数 y */
    addBigNum(z,x,y);  /* z=x+y */
    putBigNum(z);      /* 输出z */
    putchar('\n');
    return 0;
}
```



输入

- ✓ 用字符一位一位输入，先输入高位再输入低位
- ✓ 放于数组中，从**0**号元素开始放，即高位放于低地址
- ✓ 实际存储：低位在低地址（**也可以**低位在高地址）
- ✓ **反转数组元素**

函数getBigNum

- **函数功能：** 输入一个大整数，放于数组中，数组的每个元素存放一个十进制数，低位在低地址，即个位在x[1]，十位在x[2],...，x[0]放总位数。
- **函数参数：**
 - 存放大整数的数组x
 - 能输入的最多位数lim
- **函数返回值：**
void
- **函数原型**
void getBigNum(int *x,int lim);

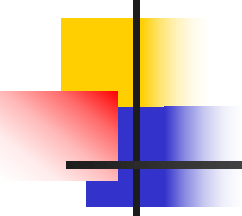
函数getBigNum

```
/*输入一个最多 lim位的大整数，存于x指向的单元，
x[0]保存总位数，个位在x[1]、十位在x[2] ,...*/
void getBigNum( int *x,int lim)
{
    int i,t,c;
    int *p1,*p2;
    for(i=1;i<=lim;i++) *(x+i)=0;        /* 先清零 */
    for(i=1;i<=lim && isdigit(c=getchar());i++)/*最先输的高位在x[1],...*/
        *(x+i)=c-'0';
    *x=i-1;        /* x[0]保存整数的位数 */
    for(p1=x+1,p2=x+i-1;p1<p2;p1++,p2--) /* 反转，低位在x[1]... */
        t=*p1,*p1=*p2,*p2=t;
}
```

函数putBigNum

- 从高位按运算结果的实际位数输出每一位（数组元素）

```
/* 输出x指向的整数 */  
void putBigNum(int *x)  
{  
    int *p;  
    int n=*x;    /* 整数的位数 */  
    for(p=x+n;p>x;p--) { /* 从高位开始输出 */  
        printf("%d",*p);  
    }  
}
```



运算过程

模拟列竖式计算两数相加（如**45+96**）。注意：

（1）运算顺序：从低位向高位运算；先低位后高位；

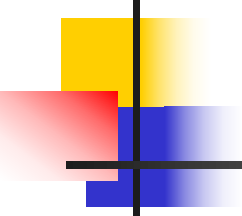
（2）运算规则：相同位的两个数相加再加上进位，成为该位的和；这个和去掉向高位的进位就成为该位的值；如上例：

5+6+0=11，向前进一位**1**，本位的值是**1**；可借助%、/运算完成这一步；

4+9+1=14，向前进一位**1**，本位的值是**4**

（3）最后一位的进位：如果完成两个数的相加后，进位位值不为**0**，则应添加一位；

（4）如两个加数位数不一样多，则按位数多的一个进行计算；



函数addBigNum

- 函数功能：两个大整数相加

- 函数参数：

 - 输入 **x**--被加数

 - y**--加数

 - 输出 **z**--和数

- 函数返回值：

 - void**

- ⑩ 函数原型

 - void addBigNum(int *z,int *x,int *y);**

函数addBigNum

/* 将x指向的被加数与y指向的加数相加, 结果放到z指向的单元 */

```
void addBigNum(int *z,int *x,int *y)
```

```
{
```

```
    int i,carry,n;
```

```
    for(i=1;i<=N;i++) /* 和数清零 */
```

```
        *(z+i)=0;
```

```
    n = max(*x,*y); /* n为两个加数中较长的位数 */
```

```
    for(carry=0,i=1;i<=n;i++) {
```

```
        *(z+i) = *(x+i) + *(y+i) + carry; /* 带进位的加法运算 */
```

```
        carry = *(z+i) /10; /* 计算新的进位 */
```

```
        *(z+i) %= 10; /* 计算本位 */
```

```
    }
```

```
    if(carry) /* 最后一位的进位 */
```

```
    {
```

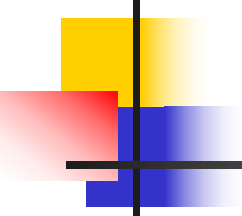
```
        *(z+i)=carry;
```

```
        i++;
```

```
    }
```

```
    *z = i-1; /* z[0]保存和数的实际位数 */
```

```
}
```



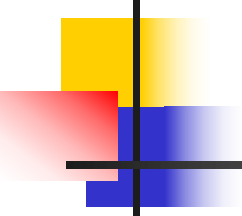
分治法

高精度计算是基于分治法。

分治法是一种很重要的算法。“分而治之”就是把一个复杂的问题分成两个或更多的相同或相似的子问题，再把子问题分成更小的子问题……直到最后子问题可以简单的直接求解，原问题的解即子问题的解的合并。这个技巧是很多高效算法的基础。

采用的分治策略：

将“一个大数分为多个1位整数（0~9）”。

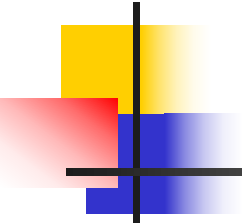


该分治策略的缺点

- (1) **浪费空间**：一个整型数组的每个元素（-32768~32767）只存放一位（0~9）；
- (2) **浪费时间**：一次加法只处理一位；

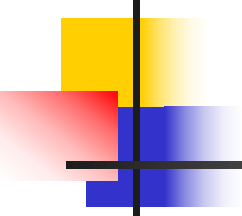
优化

- (1) 分治策略：将 “一个大数分为多个4位整数 (0~9999)” (考虑2B整数的最大范围是32767, 5位的话可能导致出界)
- (2) 一个数组元素存放四位数；将标准数组改为紧缩数组。
- (3) 运算时：逢万进位；将 “十进制运算” 改为 “万进制运算”。
- (4) 输出时：最高位直接输出，其余各位，要判断是否足够4位，不足部分要补0；例如：1, 23, 2345这样三段的数，输出时，应该是100232345而不是1232345。



思考练习

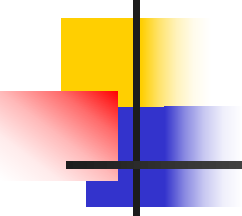
- 编程计算 $n!$ (n 可达100)



8.4 指针和字符串

字符串是一维字符数组，要访问和操纵字符串，**可以用字符数组，也可用字符指针**。欲用指针变量访问字符串，需：

- 声明字符指针变量；
- 通过初始化或者赋值使字符指针变量指向字符串；
- 通过字符指针来访问字符串中对应的字符。



字符串的指针表示

(1) 声明一个字符指针变量和一个字符数组

```
char s[80]= "This is a string." ;
```

```
char *p=s;
```

```
puts(p);
```

输出: *This is a string.*

```
putchar(*p);
```

输出: *T*

```
s="This is a book.";
```

错, *数组名s是指针常量*

```
strcpy(s, "This is a string.");
```

✓

字符串的指针表示

(2) 声明和初始化一个字符指针变量

```
char *p=" This is a string." ;
```

/* 分配内存给字符指针p;

分配内存（静态区）给字符串常量"..."（只读）；

将字符串首地址赋值给字符指针； */

```
puts(p);    /* 输出 This is a string. */
```

```
putchar(p[1]);    /* 输出 h */
```

```
p="This is a book."; /* ✓, p是指针变量, 不提倡 */
```

```
*p='t';        /* 运行错 */
```

字符串作函数参数

```
/* 字符串拷贝 */  
void mystrcpy(char *t, const char *s)  
{  
    while( (*t=*s) != '\0' )  
        t++,s++;  
}
```

```
void mystrcpy(char *t,constchar *s)  
{  
    while( (*t++=*s++) != '\0' ) ;  
}
```

```
void mystrcpy(char *t,constchar *s)  
{  
    while( *t++=*s++ ) ;  
}
```

```
char t[30],s[ ]="world";  
char *p;
```

```
mystrcpy(t,"hello"); ✓
```

```
mystrcpy(t,s); ✓
```

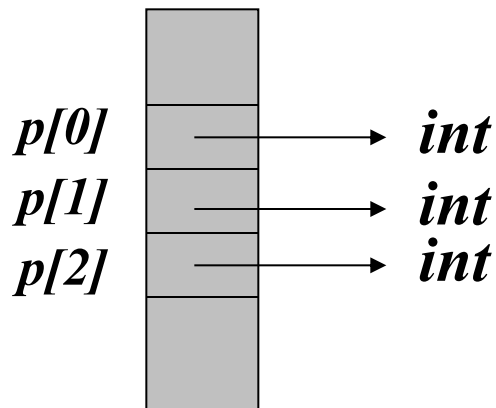
```
mystrcpy(p,s); ✗
```

```
p=t;
```

```
mystrcpy(p,s); ✓
```

8.5 指针数组

`int *p[3];` /* *p*是一个有 3 个元素的整型指针数组
即每个元素是指向整型变量的指针 */



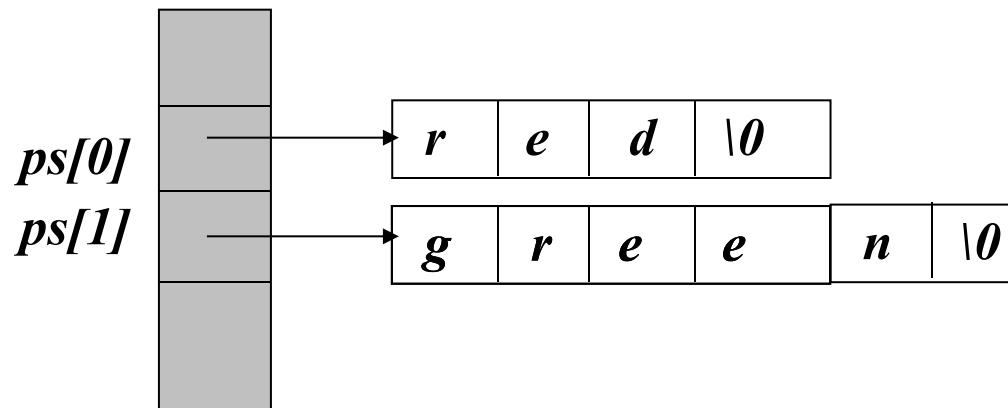
字符指针数组

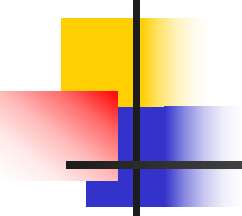
```
char *ps[2]={ “red”, “green” };
```

/ ps是一个有 2 个元素的字符指针数组*

ps[0]指向字符串 “red”

*ps[1]指字符串 “green” */*





指针数组的使用

指针数组的应用：

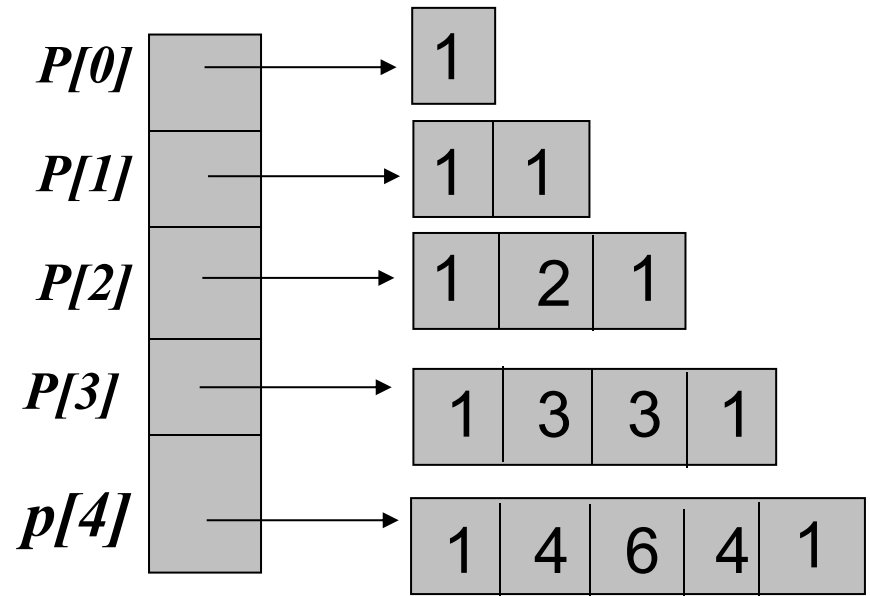
描述二维数组 (数值型二维数组, 字符串数组)

尤其是每行元素个数不相同的二维数组

(如三角矩阵, 有不同长度的字符串组成的字符串数组)

指针数组描述杨辉三角形

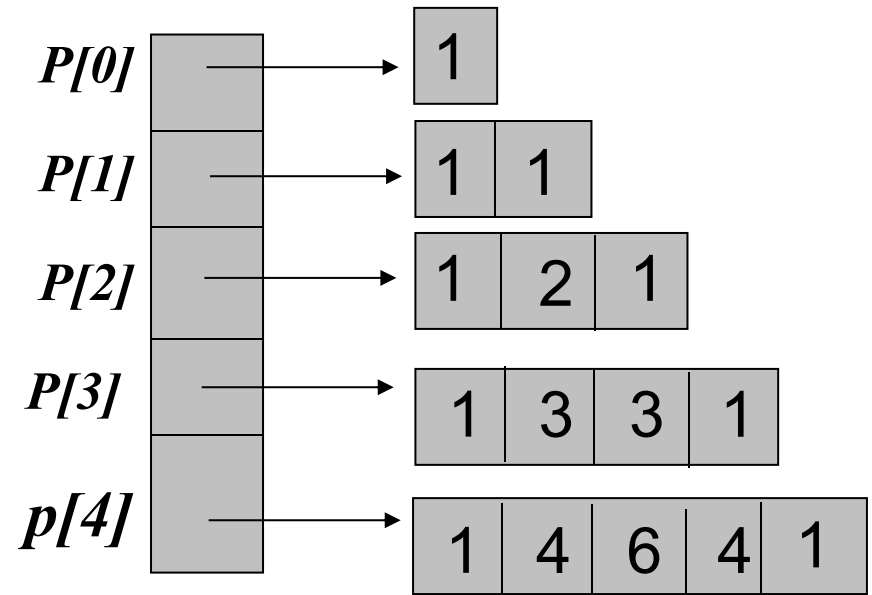
```
#define N 5
#define SIZE N*(N+1)/2
int main()
{
    int *p[N], a[SIZE];
    int sum=0,i;
    for(i=0;i<N;i++) {
        p[i]=&a[sum];
        sum += i+1;
    }
    for(i=0;i<N;i++) {
        *p[i]=1;
        *(p[i]+i) = 1;
    }
}
```

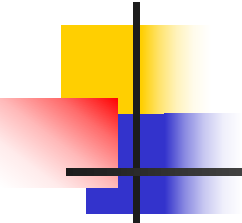


```
for(i=2;i<N;i++)
    for(k=1;k<i;k++)
        p[i][k]=p[i-1][k-1]+p[i-1][k];
return 0;
}
```

指针数组描述杨辉三角形

```
#define N 5
#define SIZE N*(N+1)/2
int main()
{
    int *p[N], a[SIZE] , i;
    for(i=0;i<N;i++) {
        p[i]=&a[ i*(i+1)/2 ];
    }
    .....
}
```





但用指针方式存取数组元素比用下标速度快，
并节省存储空间，

int *p[6], a[21] ;

$6*4+21*4=108\text{B}$

int x[6][6] ;

$36*4=144\text{B}$

菜单函数---- selectMenu

```
int selectMenu(void ) //函数返回值是整形，代表所选的菜单项
{
    static char *menu[]={
        "1:Enter record",
        "2:Search record on r",
        "3>Delete a record",
        "4:Add a record",
        "0:Quit",
        NULL
    };
    int i,ch;
    system("cls");    /* 清屏 */
    do {
        for(i=0;menu[i]!=NULL;i++)
            puts(menu[i]);
        printf("\n Enter your choice:");
        scanf("%d",&ch);    /* 输入选择项 */
    } while( ch<0 || ch>i ); /* 选择项不在1-5之间重输 */
    return ch; /* 返回选择项，调用函数根据该数调用相应的函数 */
}
```

NULL指针（空指针）
： 值为0的指针

菜单程序框架

```
#include<stdio.h>
#include<stdlib.h>
int selectMenu(void);
int main(void)
{
    int choice;
    while((choice=selectMenu())){
        switch(choice) {
            case 1: // 功能1
                break;
            case 2: // 功能2
                break;
            case 3: // 功能3
                break;
            case 4: // 功能4
                break;
        }
    }
    return 0;
}
```

判关键字函数-----iskey

NULL指针

```
/* 若s是关键字, 函数返回1; 否则, 返回0 */
int iskey(char *s)
{
    static char *keyword[]={ /* 关键字表 */
        "auto", "_Bool", "break", "case", "char", "_Complex",
        "const", "continue", "default", "restrict", "do", "double",
        "else", "enum", "extern", "float", "for", "goto",
        "if", "_Imaginary", "inline", "int", "long", "register",
        "return", "short", "signed", "sizeof", "static", "struct",
        "switch", "typedef", "union", "unsigned", "void", "volatile",
        "while", NULL};

    int i;
    for(i=0; keyword[i]!=NULL; i++) /* 将标识符s 依次与每个关键字比较 */
        if(!strcmp(s, keyword[i]))
            return 1;
    return 0; /* 否返回0 */
}
```

输入N本书名

```
#define N 3
#include<stdio.h>
int main( )
{
    int i;
    char *s[N];
    for(i=0;i<N;i++)
        fgets(s[i], 80, stdin);
    return 0;
}
```

读入的N个字符串存哪里？



如何解决



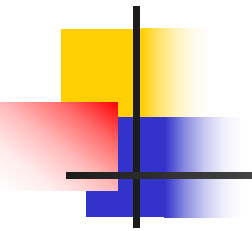
错误，使用了悬挂指针

动态分配存储字符串的空间

```
#define N 3
#include<stdio.h>
#include<stdlib.h>
int main( )
{
    int i;
    char *s[N];
    for(i=0;i<N;i++) {
        char t[80];
        fgets(t, 80, stdin);
        s[i] = (char *)malloc(strlen(t)+1);
        strcpy(s[i],t);
    }
    .....
}
```

在该头文件中给出函数原型：
void *malloc(unsigned size);

分配**size**字节的存储区。
返回所分配单元的起始地址。
如不成功，返回**NULL(空指针)**



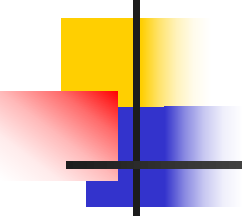
动态存储分配函数**malloc**

- 动态存储分配函数是C的标准函数，函数的原型声明在头文件**<stdlib.h>**中给出。

void * malloc(unsigned size);

功能： 分配**size**字节的存储区。

返回值： 所分配单元的起始地址。如不成功，返回**NULL**



无值型指针与空指针

类型为**void ***的指针称为无值型指针或**void**指针。不能对**void**指针执行间访操作，即对**void**指针施行“*****”操作属于非法操作。

指针值为**0**的指针称为空指针，**0**在**C**中往往用符号常量**NULL**表示并被称为空值。

```
scanf ( " %d" ,&n) ;
```

```
/* 建立大小为n的int型数组 */
```

```
p=(int *)malloc( n*sizeof(int));
```

```
if(p==NULL)  exit(-1);
```

二级指针

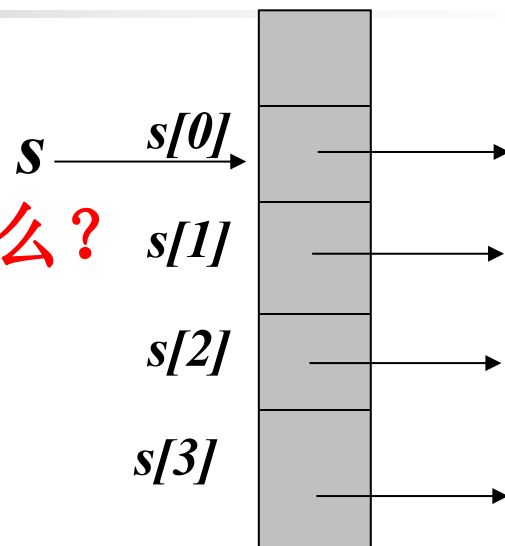
`char *s[4];`

问: `s[0][0]`, `s[0]`, `s` 类型分别是什么?

`s[0][0]` 《==》 `*s[0]` 类型 `char`

`s[0]`, 指针数组的元素, 类型 `char *`

`s`, 指针数组名, 指向 `s[0]`, 类型 `char **`



(指向字符指针的指针, 即字符型的二级指针)

`*s` 《==》 `s[0]`, `*(s+1)` 《==》 `s[1]`, `*(s+i)` 《==》 `s[i]`

`*(*(s+i)+j)` 《==》 `s[i][j]`

二级指针的应用: 作函数的形参

字符串排序函数----*strsort*

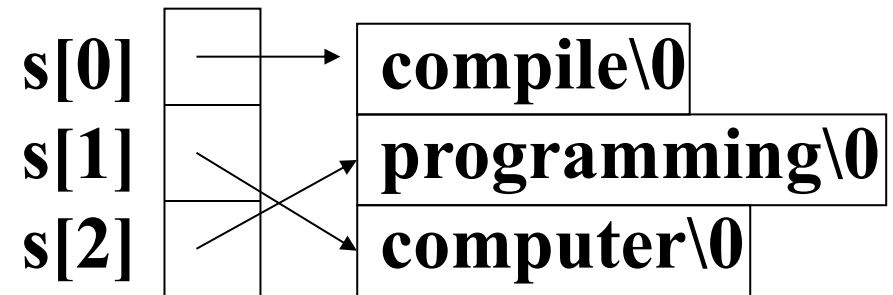
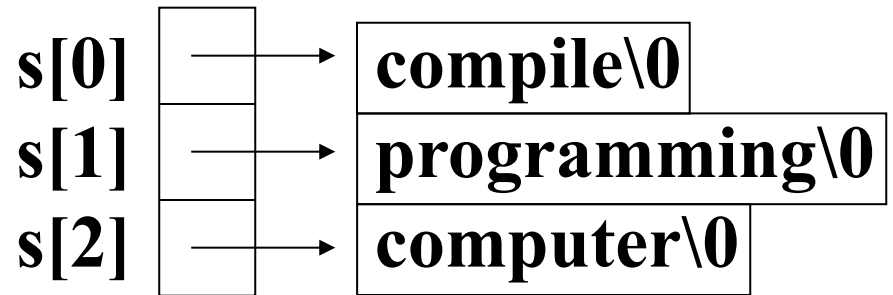
```
void strsort ( char *s[ ] , int n )  
{  
    char **s  
    char *temp;  
    int i, j ;  
    for(i=0; i<n-1; i++)  
        for(j=0; j<n-i-1; j++)  
            if ( strcmp(s[j],s[j+1] ) >0 ) {  
                temp=s[j];          /* 移动指针变量 */  
                s[j]=s[j+1];  
                s[j+1]=temp;  
            }  
}
```

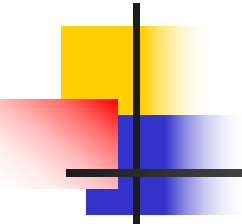
```

#define N 3
#include<stdio.h>
#include<stdlib.h>
int main( )
{
    int i;
    char *s[N], t[80];
    for(i=0;i<N;i++) {
        fgets(t,80,stdin);
        s[i] = (char *)malloc(strlen(t)+1);
        strcpy(s[i],t);
    }
    strsort(s,N);
    for(i=0;i<N;i++) puts(s[i]);
}

```

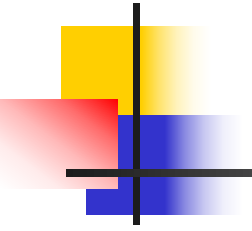
例 输入N本书名，排序后输出。





字符串排序函数----*strsort*

```
void strsort ( char **s , int n )
{
    char *temp;
    int i, j ;
    for(i=0; i<n-1; i++)
        for(j=0; j<n-i-1; j++)
            if ( strcmp(*(s+j),*(s+j+1)) >0 ) {
                temp=*(s+j);    /* 移动指针变量 */
                *(s+j)=*(s+j+1);
                *(s+j+1)=temp;
            }
}
```



8.6 带参数的main函数

求 $1+2+\dots+n$ ， n 值由用户指定

方式1：运行后输入

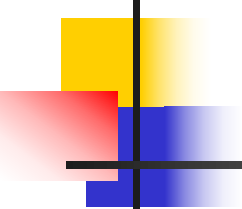
方式2：命令行中输入

程序执行时所需的数据何时输入

命令行：在命令行界面（如Windows的cmd.exe）下，用户输入的用于运行程序的行。

命令行参数：在命令行中空格隔开的字符串。

运行程序时操作系统将命令行参数传给main的形数。



方式1的main函数

```
#include<stdio.h>
```

```
int main(void)
{
    int n,sum,i;
    scanf("%d",&n);
    for(sum=0,i=1;i<=n;i++)
        sum+=i;
    printf("1+2+...+%d=%d\n",n,sum);
    getchar();
    return 0;
}
```

方式2的main函数

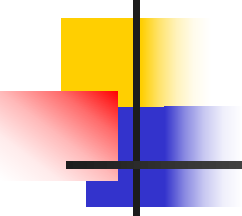
C:\> **sum 11**

```
char *argv[]={  
    "sum",  
    "11",  
    NULL  
};
```

```
#include<stdio.h>  
#include<stdlib.h>  
int main(int argc, char *argv[])  
{  
    int n, sum, i;  
    if(argc!=2) {  
        printf("Command error\n");  
        return -1;  
    }  
    n=atoi(argv[1]);  
    for(sum=0, i=1; i<=n; i++)  
        sum+=i;  
    printf("1+2+...+%d=%d\n", n, sum);  
    return 0;  
}
```

长度为argc的字符指针数组
每个参数是一个字符串，
有argc个字符串

命令行中参数的个数（包括文件名）



带参main函数的一般形式

```
int main(int argc, char *argv[])  
{  
    ...  
}
```

实参由命令行提供。

argc: argument count,

代表命令行中参数的个数（包括文件名）

argv: argument value, 为字符指针数组，

argv[i]指向命令行的第 **i+1** 个参数（字符串）的
首字符

命令行参数的传递

C:\> sum 11

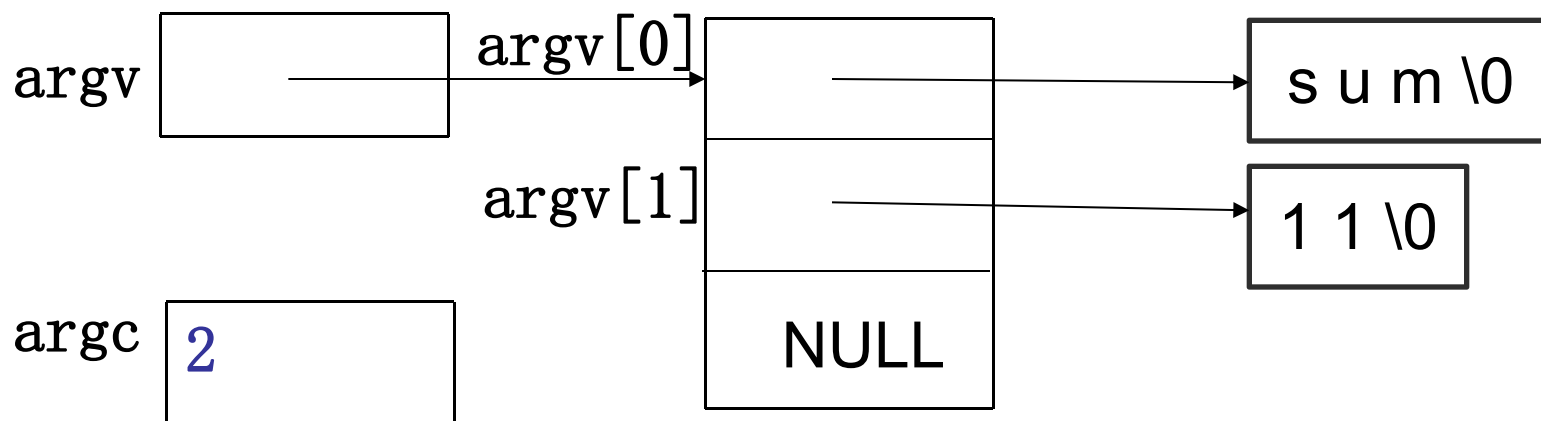
```
int main(int argc, char *argv[])
```

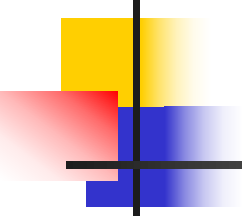
```
{
```

```
.....
```

```
}
```

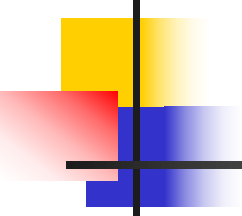
char **argv





argv的类型实质为 char **

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char **argv)
{
    int n,sum,i;
    if(argc!=2) {
        printf("Command line error!\n");
        return 1;
    }
    n=atoi(*++argv);
    .....
}
```

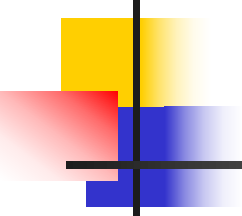


指定main的参数

在集成开发环境下调试程序时命令行所带参数如何输入？(只输入文件名后的参数，文件名不输入)

见 实验指导书

- | | | | |
|-----|-------------|-----|---------|
| (1) | CodeBlocks: | P7 | 1. 1. 7 |
| (2) | Dev: | P10 | 1. 2. 6 |
| (3) | VS: | P18 | 1. 3. 6 |



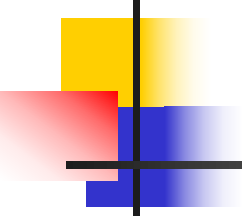
8.7 指针函数

在C语言中，函数返回的只能是值。这个值可以是一般的数值，也可以是某种类型的指针值。如果函数的返回值是指针类型的值，该函数称为指针函数。

类型 *函数名（形参表）；

如：char *strcpy(char *t, const char *s)；

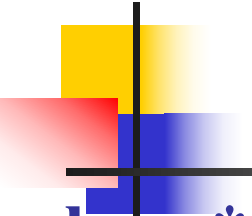
函数strcpy是一个字符指针函数。即：该函数的返回值是字符指针。



指针函数的定义

```
char *strcpy( char *t, const char *s )
{
    char *p=t ;
    while(*t++ = *s++);
    return(p) ; /* 返回第1个串的首地址 */
}

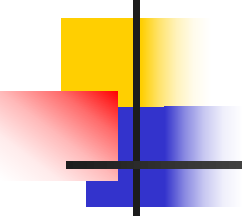
int main( )
{
    char st1[40]=" abcd" , st2[ ]=" hijklmn" ;
    printf( "%s" , strcpy( st1,st2));
    return 0;
}
```



指针函数的定义

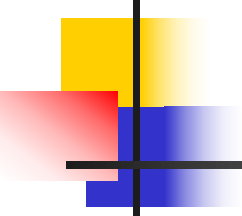
```
char *strcpy( char *t, const char *s )
{
    char *p=t ;
    while(*t++ = *s++);
    return(p) ; /* 返回第1个串的首地址 */
}

int main( )                                // 哪里有错
{
    char st1[ ]=" abcd" , st2[ ]=" hijklmn" ;
    printf( "%s" , strcpy( st1,st2));
    return 0;
}
```



查找子串的指针函数

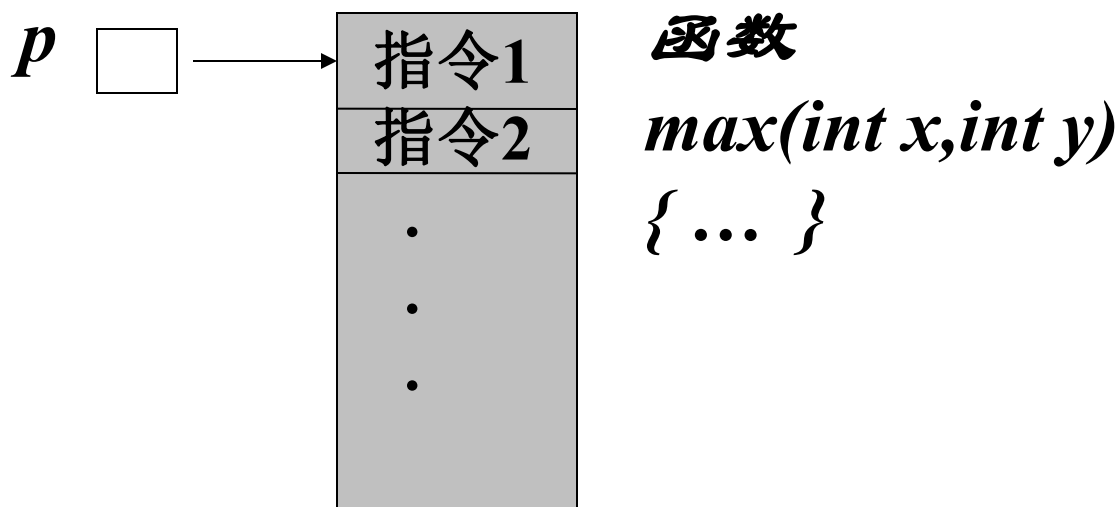
```
char *strstr ( const char *s,const char *ms)
{ char *ps=s,*pt,*pc;
  while(*ps!='\0'){
    for(pt=ms,pc=ps;*pt!='\0'&&*pt==*pc; pt++, pc++)
      ;
    if(*pt=='\0') return ps;
    ps++;
  }
  return NULL; /* NULL : 0 */
}
```



如果函数返回的指针指向一个数组的首元素，就间接解决了函数返回多值的问题。

8.8 函数的指针

每个函数都占用一段内存单元，有一个起始地址。



如何声明 p ?

函数指针的声明

类型 (*标识符) (形参表);

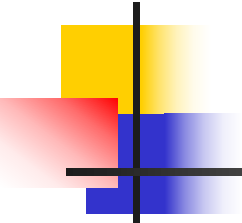
↑ ↑ ↑
所指函数的返回类型 函数指针名 所指函数的形参的类型与个数

*int (*p) (int, int);*

/ p 是指向有两个 int参数的int函数的指针 */*

函数指针的使用:

- (1) 通过初始化使其指向特定的函数;
 函数指针名=函数名;
- (2) 通过函数指针来调用它所指的函数



函数指针的使用

```
void f1(int x)
```

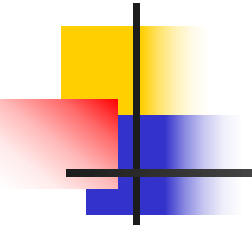
```
{  
    printf("x=%d\n",x);  
}
```

```
void f2(int x,int y)
```

```
{  
    printf("x=%d\n",x);  
    printf("y=%d\n",y);  
}
```

```
int main(void)
```

```
{  
    void (*pf1)(int x);  
    void (*pf2)(int x,int y);  
    pf1=f1;  
    pf2=f2;  
    pf1(5); //等价于(*pf1)(5);  
    pf2(10,20); //等价于(*pf2)(10,20);  
    return 0;  
}
```



通用的整数排序函数

- 既能实现升序排序，也能实现降序排序

函数名称：**sort**

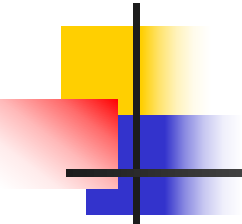
函数参数：

v--待排序数组的首地址

n--数组中待排序元素数量

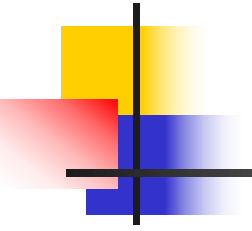
comp--指向函数的指针，用于确定排序的规则

函数返回值：无



通用的整数排序函数

```
/*对指针v 指向的n个整数按comp规则排序 */  
void sort ( int *v, int n, int (*comp)(int, int) )  
{  
    int i, j ;  
  
    for(i=1; i<n; i++)        /*冒泡法*/  
        for(j=0; j<n-i; j++)  
            /*对v[j]和v[j+1]按照comp的规则进行比较*/  
            if ( (*comp)(v[j],v[j+1]) )  
                swap( v+j, v+j+1) ;        /* 交换 */  
}
```



规则：按升序排序

//回调函数，通过函数指针参数调用的函数

```
int asc(int x, int y)
```

```
{
```

```
    if(x>y) return 1;
```

```
    else return 0;
```

```
}
```

// caller

```
int a[6]={4,6,3,9,7,2};
```

```
sort(a,6,asc);
```

// 思考：如果要降序，如何定义回调函数？

进阶：定义更通用的排序函数

- 能够对int、char、double、字符串、struct类型的数据排序。
- 既能实现升序排序，也能实现降序排序

- 函数参数

void *v, 待排序数组首地址

int n, 数组中待排序元素数量

int size, 各元素的占用空间大小（字节）

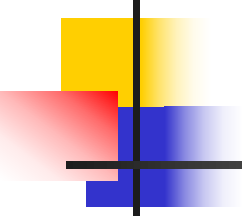
int (*fcmp)(const void *,const void *), 指向函数的指针，用于确定排序的规则

- **stdlib.h**中的标准库函数**qsort**---万能数组排序函数

**void qsort(void *base, int nelem, int width,
int (*fcmp)(const void *,const void *));**

- **P147**（快速排序）

- **思考**：如何调用**qsort**对字符串数组排序。



8.9 指针与二维数组

用指针表示二维数组元素。

- (1) 用指向数组元素的指针
- (2) 用指向数组（即行）的指针

用指向数组元素的指针表示二维数组

```
#define M 3
#define N 2
int a[M][N]={
    {1, 3},
    {4, 6},
    {7, 9}
};
int *p;
```

思考：如何用指针p逐行输出数组a的所有元素？

数组a的元素为int类型，则指向数组元素的指针应为 **int ***。

- (1) 使该指针指向一个数组元素；
- (2) 用该指针表示数组元素；
- (3) 对该指针还可以施行运算，使它指向数组中的其它元素。

```
for(p=a[0]; p<a[0]+M*N;p++){
    if (!( (p-a[0])%N ))
        printf("\n");
    printf("%5d",*p);
}
```

用指向数组元素的指针表示二维数组

```
int a[M][N]={  
    {1, 3},  
    {4, 6},  
    {7, 9}  
}, *p;
```

// 用指针p逐行输出数组a的所有元素

p=a[0]; //等价于 p=&a[0][0]

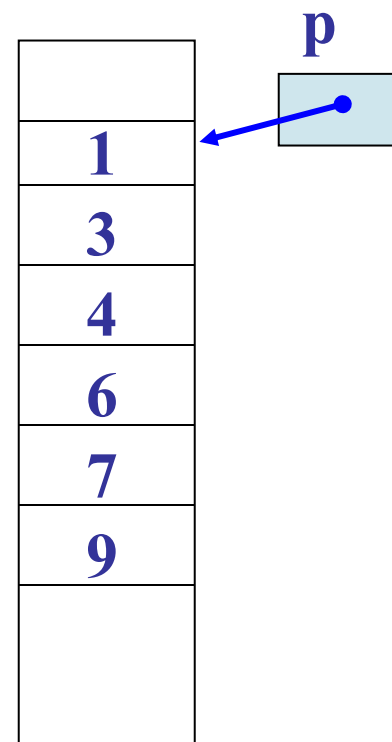
```
for(i=0; i<M;i++) {
```

```
    printf("\n");
```

```
    for(j=0; j<N;j++)
```

```
        printf("%5d", *(p+i*N+j) ); // a[i][j]
```

```
}
```



用指向数组元素的指针表示二维数组

```
int a[3][2]={
    {1, 3},
    {4, 6},
    {7, 9}
};
int *p;
```

将指针**p**指向数组的首元素

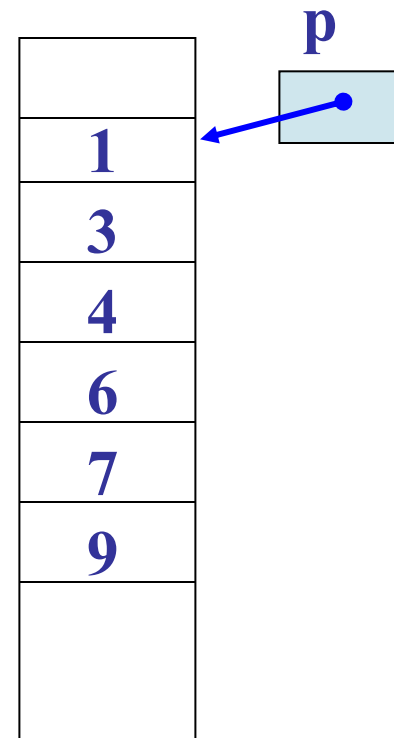
p=&a[0][0]; ✓

p=a[0]; ✓

p=a; ✗

p=(int *)a; ✓

p= *a; ✓



指针常量**a**是什么类型?

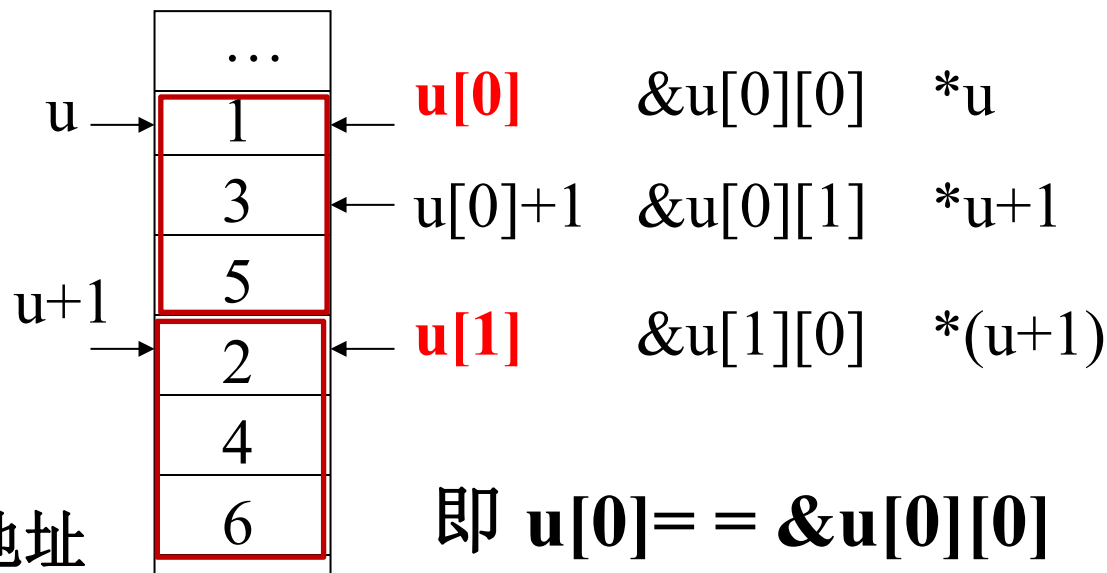
---- 指向数组的指针,即指向行的指针

int (*)[2]

行地址与元素地址

二维数组被看成以1维数组（行）为元素的数组。

```
int u[2][3]={  
    {1, 3, 5}  
    {2, 4, 6}  
};
```



`u[0]` 第0行首元素地址

`u[1]` 第1行首元素地址

`u` 第0行地址

即 `u[0] == &u[0][0]`

即 `u[1] == &u[1][0]`

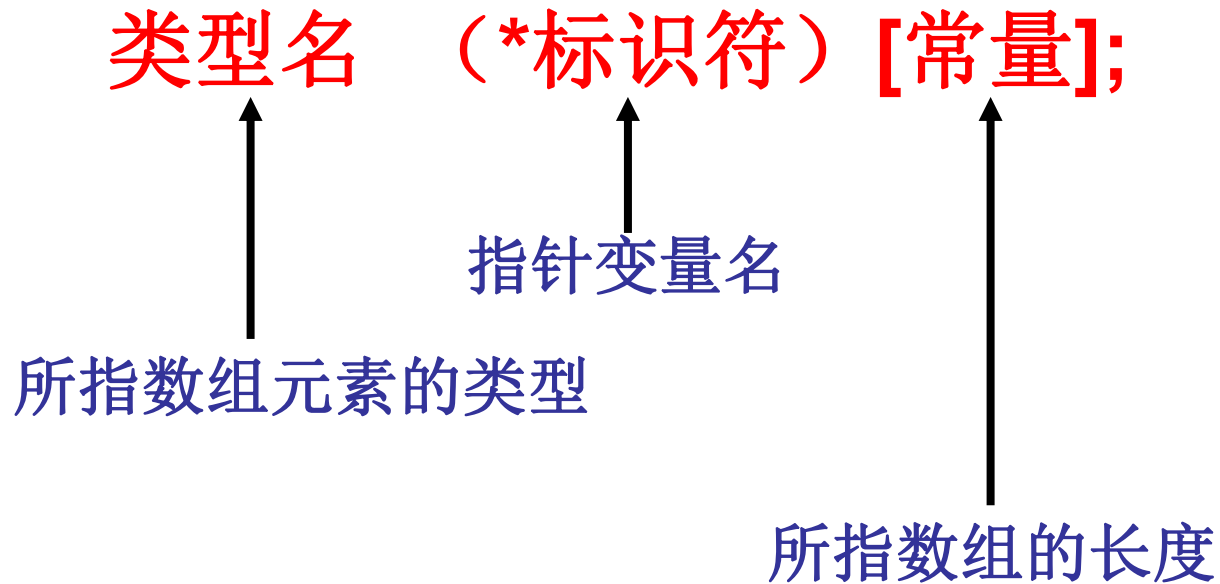
`*u` 第0行首元素地址

i行j列元素的地址: `&u[i][j]`、`u[i]+j`、`*(u+i)+j`

i行j列元素的表示: `u[i][j]`、`*(u[i]+j)`、`*(*(u+i)+j)`

指向数组的指针

指向数组的指针又称为**数组的指针**。



用指向数组的指针表示二维数组

```
int a[3][2]={ {1, 3}, {4, 6}, {7, 9} };
```

```
int (*p)[2]; // p是指向数组的指针, 该数组有2个int 型元素
```

```
p=a; // p[i][j] : a[i][j]
```

<code>*(<i>*p</i>)</code> 或 <code>(<i>*p</i>)[0]</code>	<code>*(<i>*p</i>+1)</code> 或 <code>(<i>*p</i>)[1]</code>
<code>*(<i>*(p+1)</i>)</code> 或 <code>(<i>*(p+1)</i>)[0]</code>	<code>*(<i>*(p+1)</i>+1)</code> 或 <code>(<i>*(p+1)</i>)[1]</code>
<code>*(<i>*(p+2)</i>)</code> 或 <code>(<i>*(p+2)</i>)[0]</code>	<code>*(<i>*(p+2)</i>+1)</code> 或 <code>(<i>*(p+2)</i>)[1]</code>

二维数组元素的输入/输出

```
#include <stdio.h>
#define I 2
#define J 3
int main(void)
{   int u[I][J], (*p)[J]=u;
    int j;
    for(j=0;j<J;j++)    /* 用指向数组元素的指针完成第0行元素的输入 */
        scanf("%d",(&u[0]+j));
    for(p++,j=0;j<J;j++) /* 用指向数组的指针完成第1行元素的输入 */
        scanf("%d",(&*p+j));
    for(j=0;j<J;j++) /* 用指向数组的指针完成第0行元素的输出 */
        printf("%6d",&*u+j));
    printf("\n");
    for(j=0;j<J;j++)    /* 用指向数组元素的指针完成第1行元素的输出 */
        printf("%6d",&(u[1]+j));
    return 0;
}
```

二维数组作函数参数

- 形参说明为数组
 - 形参说明为指针
- 指向数组元素的指针
指向下一级数组的指针

```
fun(int x[ ][4], int row)  
{ ... }
```

```
fun(int *x,int row,int col)  
{ ... }
```

```
fun(int (*x)[4], int row )  
{ ... }
```

```
fun(int row,int col,int (*x)[col])  
{ ... }
```

变长数组（C99）

三维数组的指针表示

类推:

三维数组被看成以二维数组（页）为元素的一维数组
一个n维数组被看成以n-1维数组为元素的一维数组。

```
int x[2][3][4];
```

```
int (*p)[3][4]=x; /*p是指向3行4列的二维整型数组的指针*/
```

```
*(*(*(p+i)+j)+k)
```

```
*(*(p[i]+j)+k)
```

```
*(p[i][j]+k)
```

```
p[i][j][k]
```

} x[i][j][k]

8.10 用typedef定义类型

typedef是关键字，为一个已有类型定义一个别名。

(1) **typedef unsigned int size_t;**

size_t被定义为 unsigned int类型

size_t x,y; /* unsigned int x,y; */

(2) **typedef char *string ;**

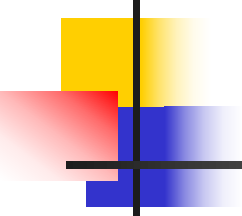
string 被定义为 char *

string p, s[10]; /* char *p, *s[10]; */

(3) **typedef int (*p_to_fun)(int ,int);**

p_to_fun 被定义为 int (*)(int ,int)

p_to_fun pf; /* int (*pf)(int ,int) */



typedef与#define的区别

```
#define string char *
```

/ string 是宏名，简单的串替换 */*

```
string p, s[10];
```

/ char *p, s[10];*

p是字符指针,

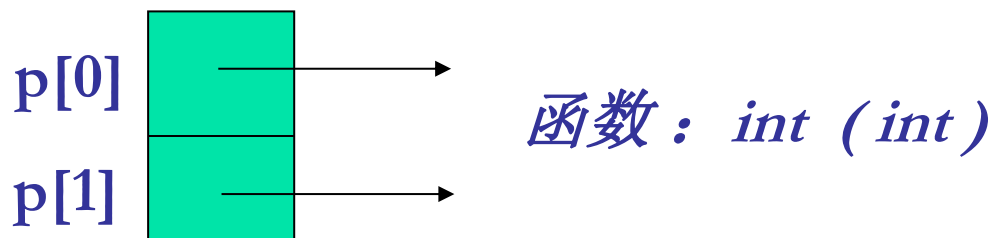
*s 是含有10个元素的字符数组 */*

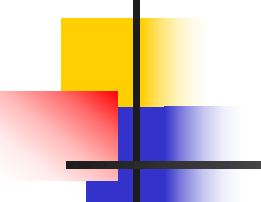
8.11 复杂声明

*int (*p[2]) (int);*

p是含有2个指针元素的数组，每个指针指向有一个整型参数，返回值为整型的函数。

p是指向函数的指针数组。





函数指针数组的使用

```
int fun1(int x)
{ return 2*x; }
```

```
int fun2(int y)
{ return 3*y; }
```

```
int main(void)
{ int i,(*fp[2])(int); /* fp 是有2个元素的函数指针数组,
                        每个元素指向的函数有一个整型形参, 返回整型值*/
  fp[0]=fun1;          /* 0号元素指向 fun1函数 */
  fp[1]=fun2;          /* 1号元素指向 fun2 函数 */
  for(i=0;i<2;i++)
    printf("%d\n",fp[i]((i+1)*5)); /*依次调用 fp[0]、fp[1]所指函数 */
  return 0;
}
```

函数指针的应用----多分支函数处理

- 函数指针数组可用于设计所谓的派遣（dispatch）表，以**实现多分支函数处理问题**，从而省去了大量的if或switch语句。如：设有10个函数 $f_0(x), f_1(x), \dots, f_9(x)$ 以及一个变量index。当 $\text{index}=i$ 时，就调用 $f_i(x)$ 。
- 方法：对函数 $f_0(x), f_1(x), \dots, f_9(x)$ 进行声明，再定义一个函数指针数组，其元素分别指向各函数。

double f0(int), f1(int), ..., f9(int); // 函数原型

static double (*dispatch[])(int) = {f0, f1, ..., f9}; // 函数指针数组

- 要调用变量index所指定的函数，可写成如下形式：

dispatch[index](x); // (*dispatch[index])(x);

- 采用该方法编写的源程序，结构清晰、易读、易修改。更具有C语言的风格。

用函数指针数组控制菜单的驱动

```
void (*cmd[4])(void)={ f1, f2, f3, f4 };
```

/ cmd 是有4个元素的函数指针数组, 函数无返回值, 无参数 */*

```
int main( void )
```

```
{ .....
```

```
do {
```

```
    printf( " menu1 \n " );
```

```
    printf( " menu2 \n " );
```

```
    printf( " menu3 \n " );
```

```
    printf( " menu4 \n " );
```

```
    printf( " exit(0) \n " );
```

```
    printf( " Enter your choice: \n " );
```

```
    scanf( " %d " ,&choice);
```

```
    if(choice>=1&&choice<=4)
```

```
        cmd [choice-1]( );
```

```
    } while(choice);
```

```
}
```

```
void f1( )
```

```
{ ... }
```

```
void f2( )
```

```
{ ... }
```

```
void f3( )
```

```
{ ... }
```

```
void f4( )
```

```
{ ... }
```

指向函数的指针函数

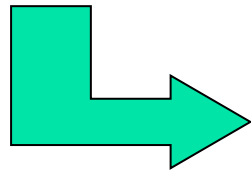
```
int (*f(char *, char *)) (int, int) ;
```

f是一个指针函数，

f函数有2个char * 类型的形参，其返回值是指向有2个int参数且返回值为int的函数的指针。

f函数 的形参： (char *, char *)

f是一个指针函数



函数： int (int, int)

指针函数的指针

```
char * (*(*v)(void))[10];
```

- (1) $(*v)$, v 是指针变量
- (2) $(*v)(\text{void})$, v 是指向函数的指针, 所指函数无参数
- (3) $(*(*v)(\text{void}))$, v 所指函数的返回值是指针
- (4) $(*(*v)(\text{void}))[10]$, 返回的指针指向有10个元素的数组
- (5) $\text{char } *(*(*v)(\text{void}))[10]$, 数组元素的类型是 $\text{char } *$

v 是指向函数的指针, 所指函数没有参数, 返回值是指向有10个元素的字符指针数组的指针。

